

# Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

# ISO 25010

| Software quality | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Suitable** | **Efficient** | **Compatible** | **Usable** | **Reliable** | **Secure** | **Maintainable** | **Flexible** | **Safe** |
| Complete | Time | Co-existence | recognizable | fault-free | confidential | modular | adaptable | operations |
| Correct | Resources | interoperable | learnable | availability | integrity | reusable | scalable | id risks |
| Appropriate | Capacity | | user error | fault tolerant | non-repudiation | analysable | installable | fail-safe |
| | | | engagement | recoverability | accountability | modifiable | replaceable | hazard warn |
| | | | inclusivity | | authenticity | testable | | safe integration |
| | | | assistance | | resistance | | | |
| | | | self-describe | | | | | |

# Risk

Risk = cost × likelihood

| | cheap | expensive |
|---|---|---|
| unlikely | Accept | Transfer (insure) |
| likely | Reduce | Avoid |

# Security must be built-in from the start

- Complex and subtle
  - You cannot roll-your-own, must use standard solutions
  - If hodge-podge, weakest link determines security
- Security architecture
  - Follow standard
  - Moving target
    - Solns that were secure in 2023, are not in 2025
      - e.g.: RSA public key encryption, no longer secure
- Security design patterns
  - Out-of-scope for ASD

# Path traversal ../../..



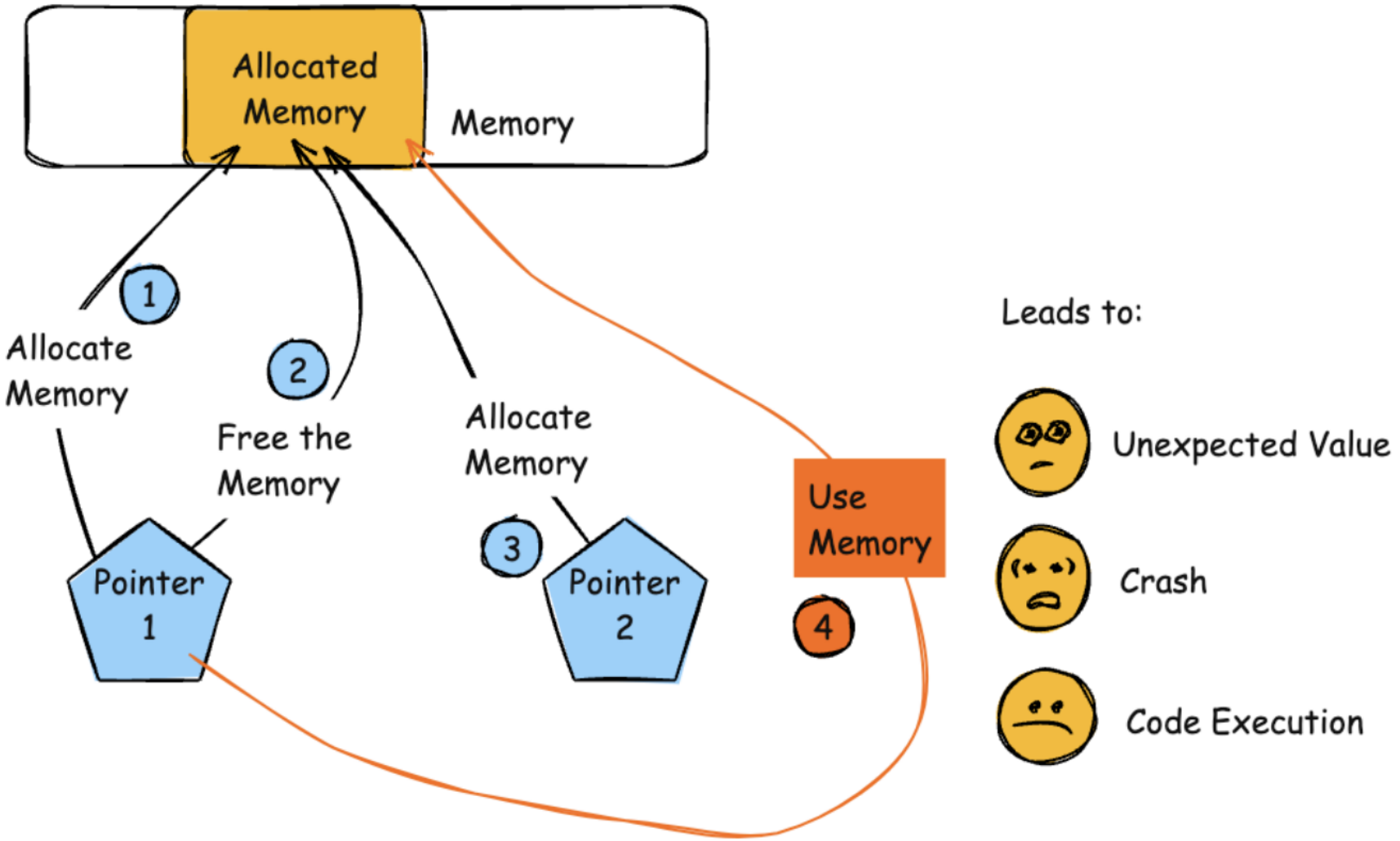
# Path traversal ../../..



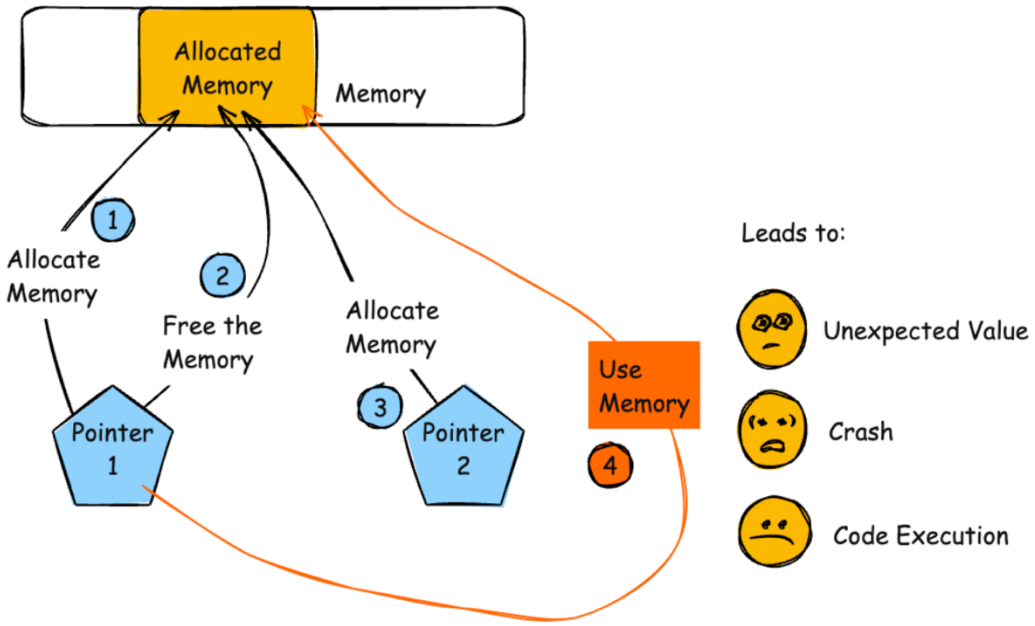

Mitigated by:

- Disallow access to special devices (COM1/NUL)
- Disallow symlink/hardlink traversal
- Normalize paths (strips out leading .., fixes middle ..)
  - Ensure normalized path is a whitelisted directory

# Use after free



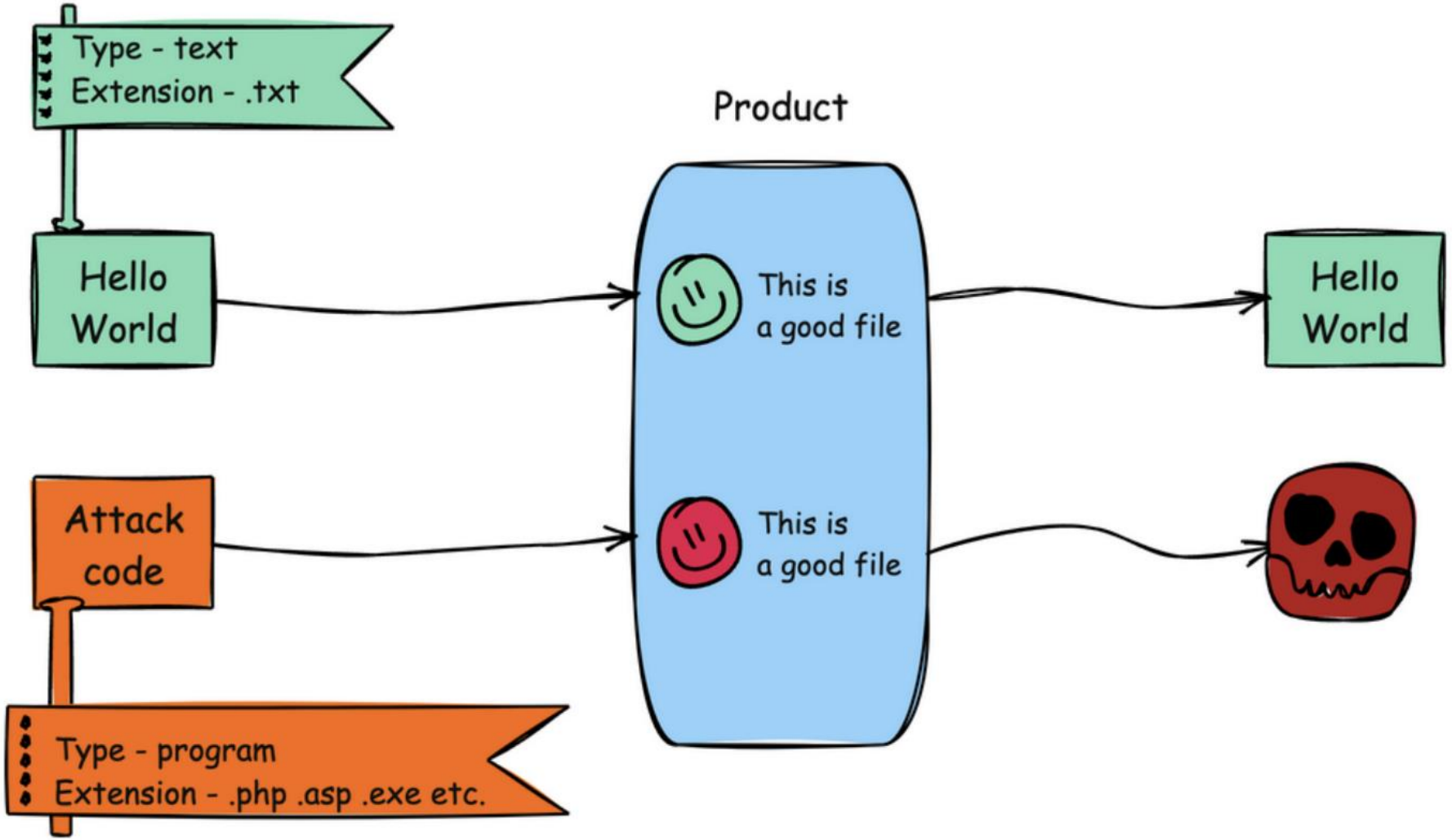
# Use after free



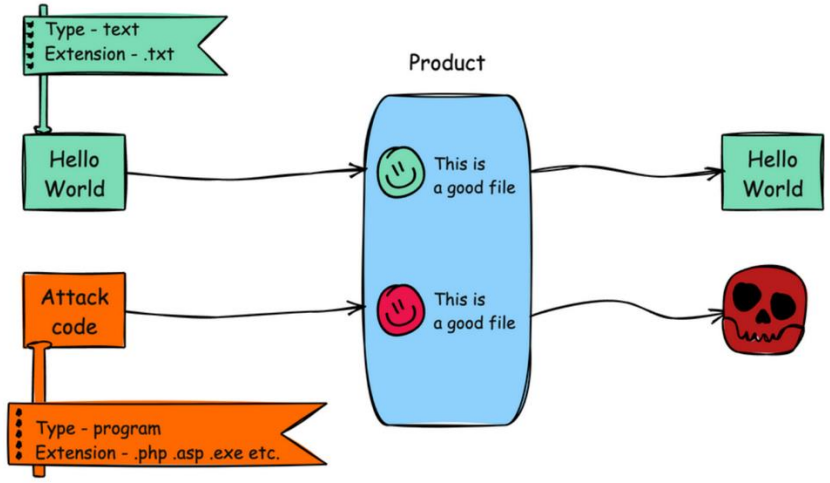

Mitigated by:

- C++/C use address sanitizer
- C++: never use `new`/`delete`
  - use `unique_ptr`/`shared_ptr`/RAII instead
- Use secure languages: Rust/Java

# Dangerous file upload



# Dangerous file upload




Mitigated by:

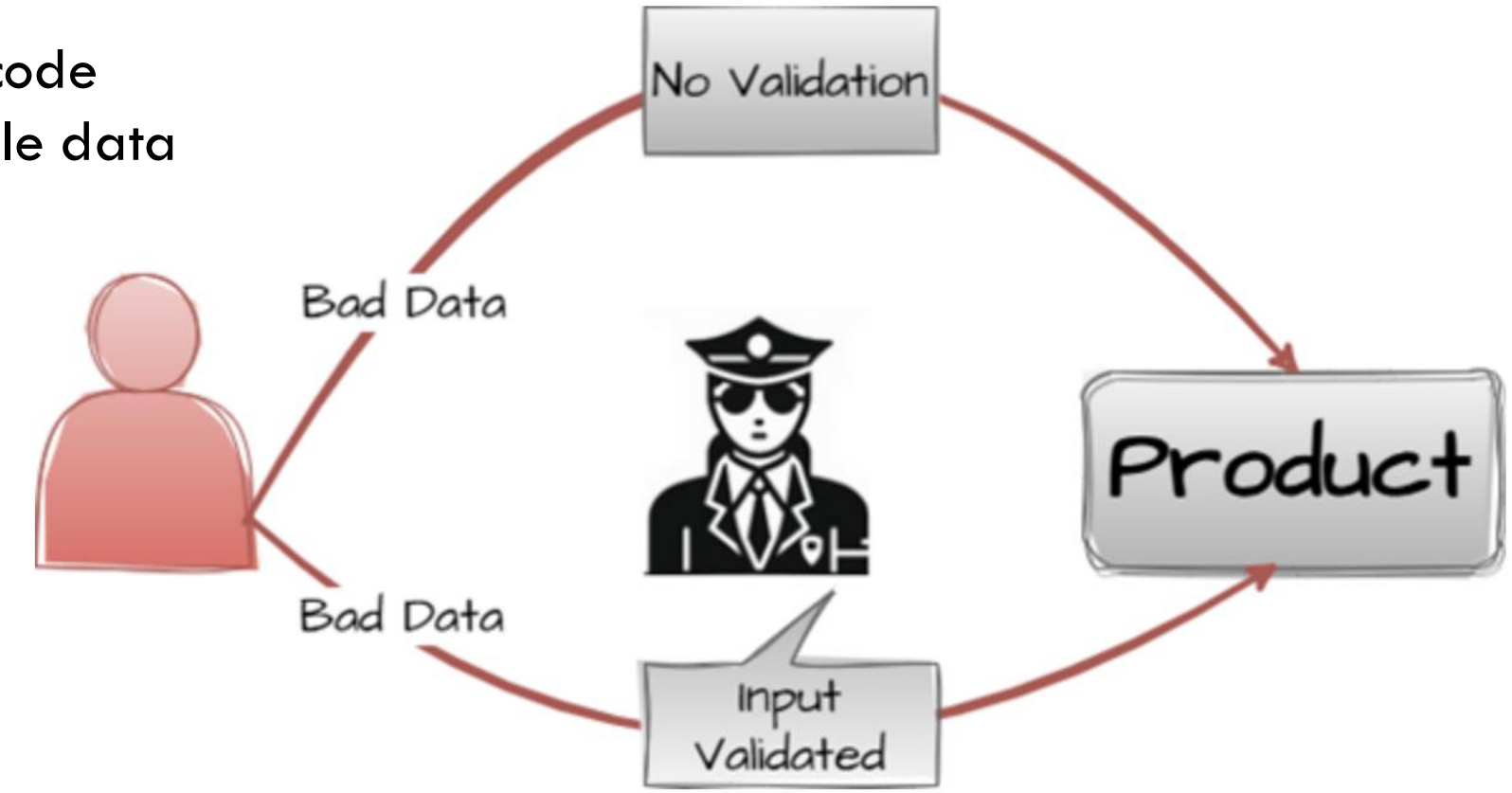
- Sanitize filenames (e.g.: utf tricks `.php%00.jpg`)
- Whitelist allowed filetypes
- Disallow overwriting existing files on the system
- Limit upload size
- Store uploads in a database, not the filesystem
- Validate allowed files (e.g.: .jpg is valid jpeg)

# Improper input validation

Similar to injection

But injection = source code

Here: machine readable data

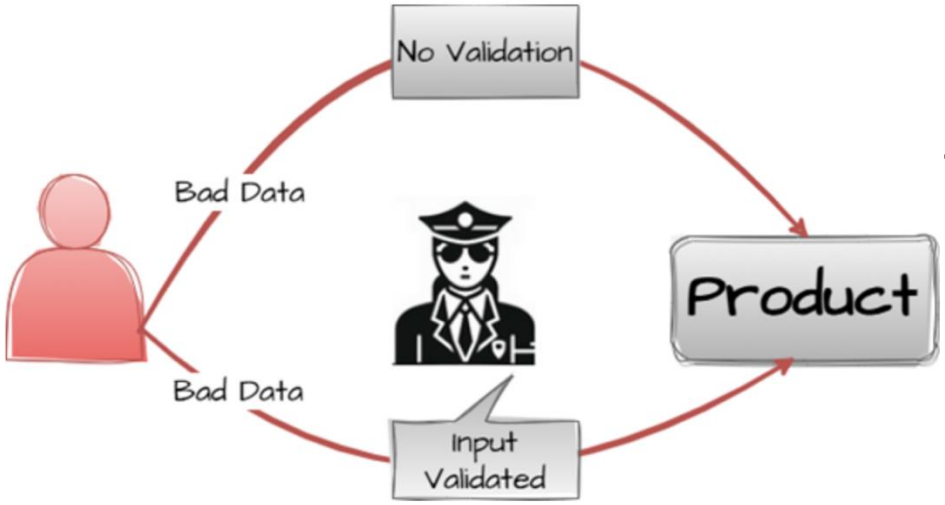


# Improper input validation

Similar to injection

But injection = source code

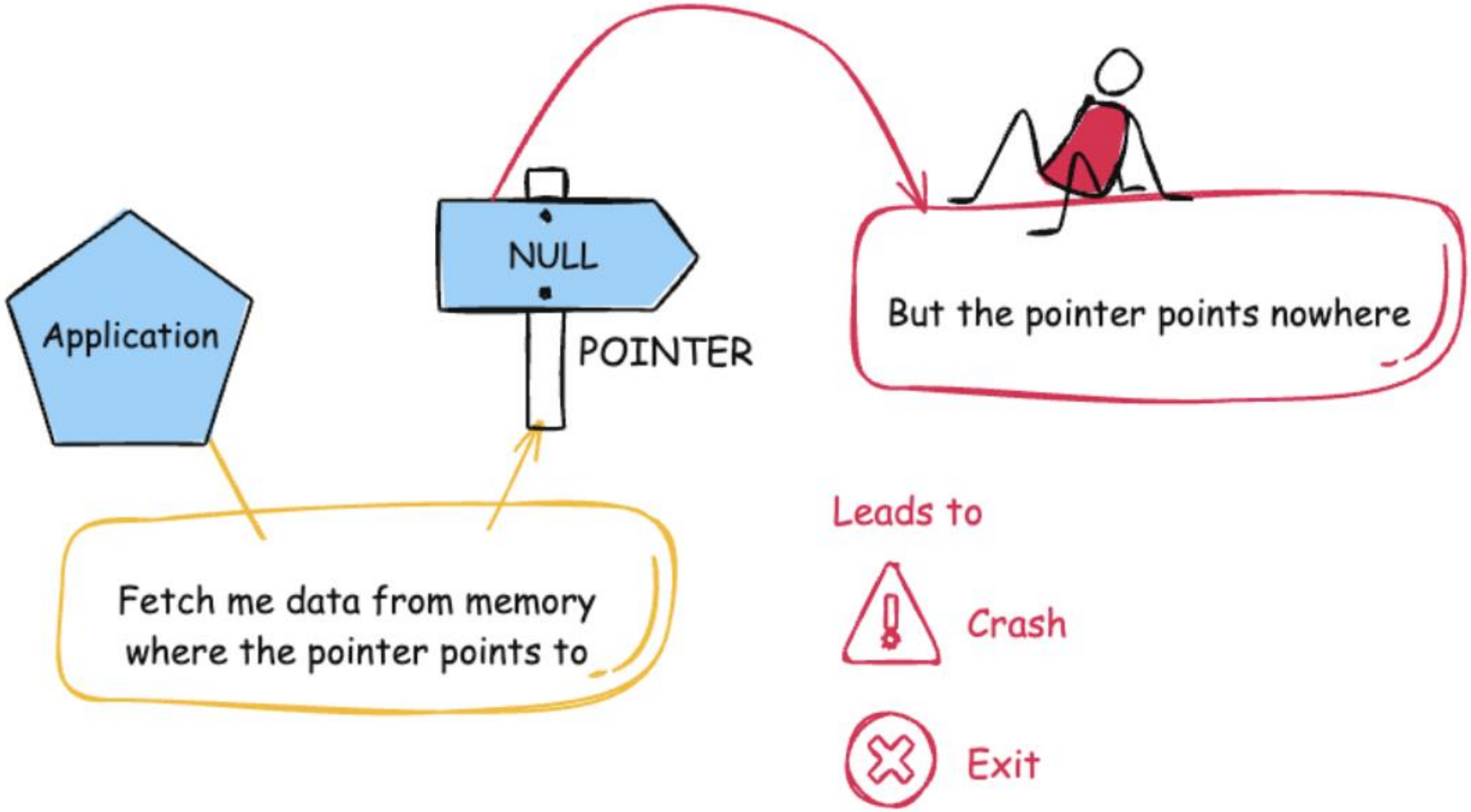
Here: machine readable data



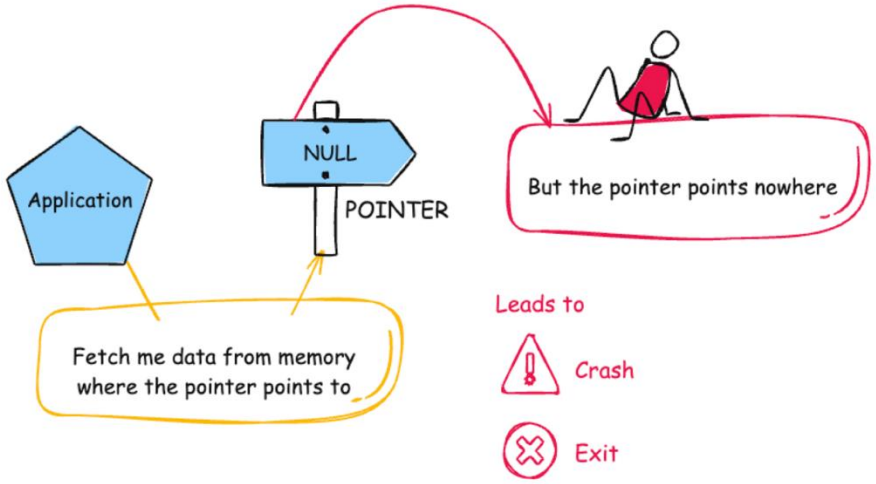

Mitigated by:

- Use known good input validators
- Use whitelist
  - Do not use a blacklist!
- Unicode (unicode sucks)
  - Normalize unicode
  - Whitelist unicode categories
    - Numbers/letters
- Do not use regex!

# NULL pointer dereference



# NULL pointer dereference

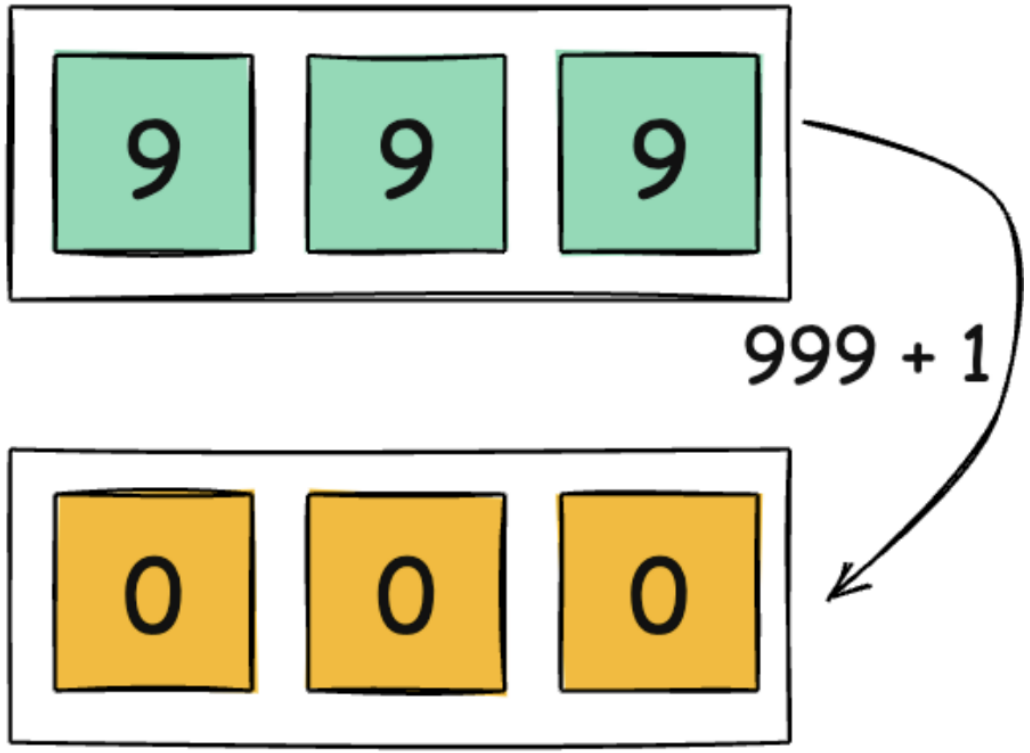



Mitigated by:

- C++/C: address/memory sanitizer
- C++: never use `new`/`delete`
  - use `unique_ptr`/`shared_ptr`/RAII instead
- Secure language: Rust/Java

# Integer overflow




Leads to:



Unexpected Value

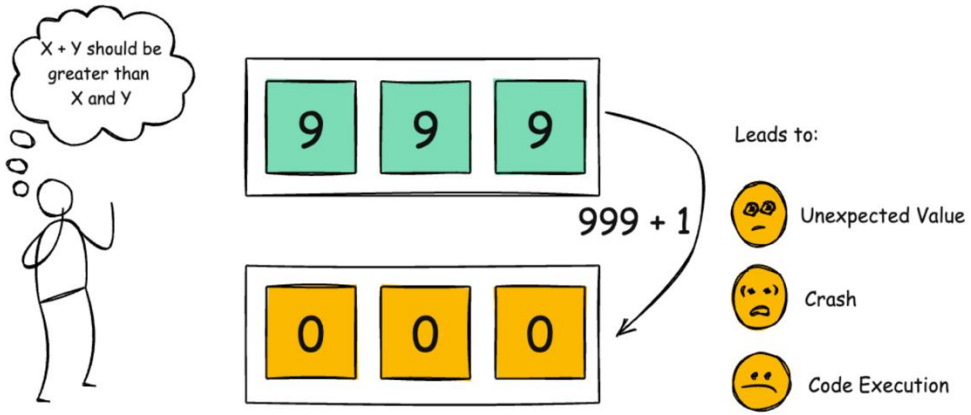


Crash



Undefined behaviour
Code Execution

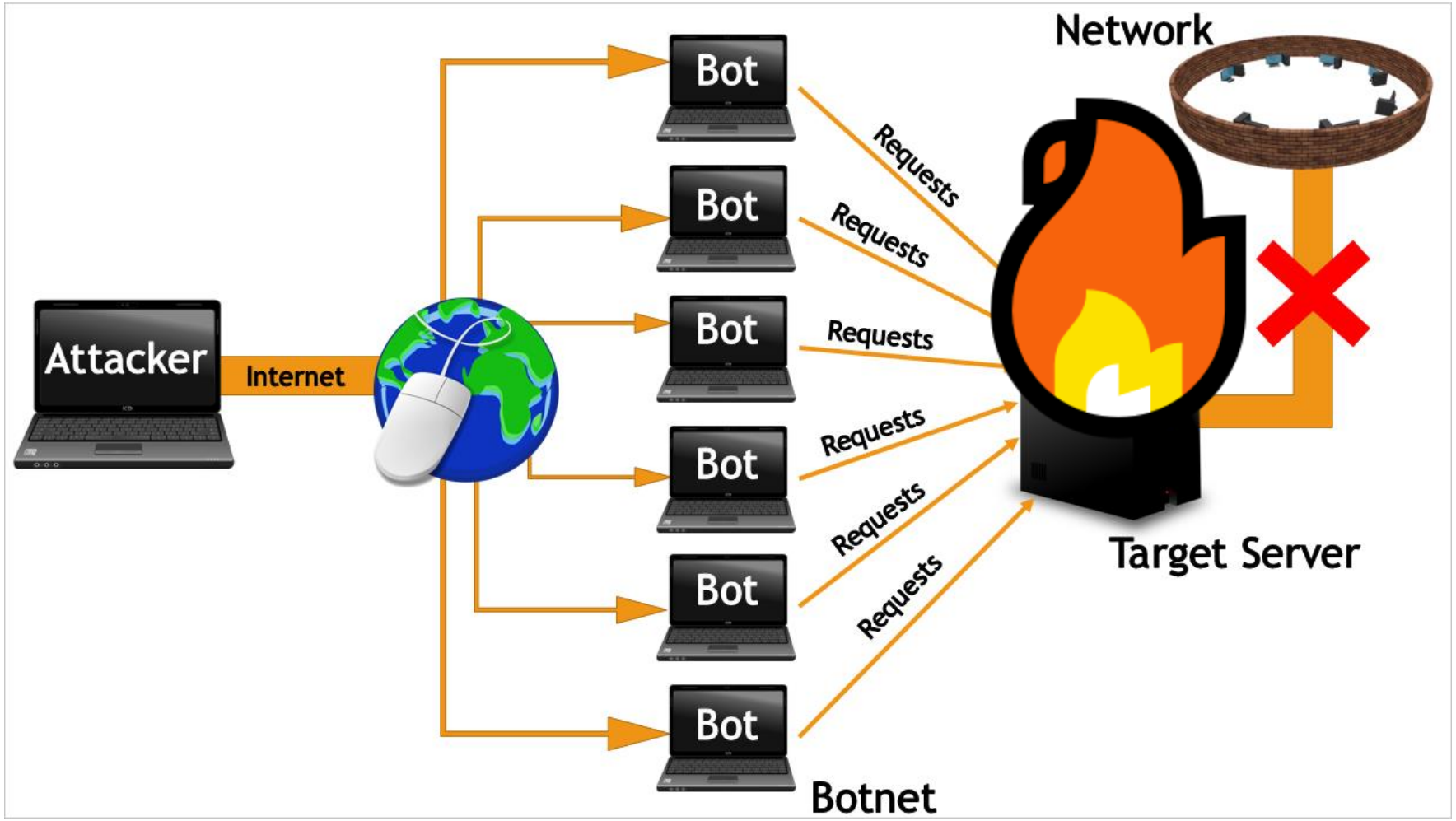
# Integer overflow



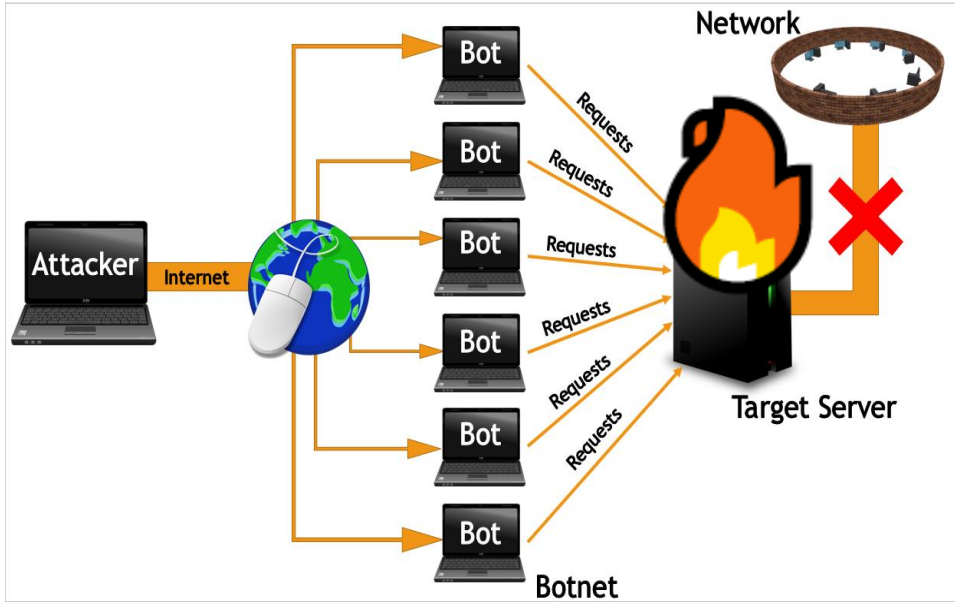

Mitigated by:

- C++/C: undefined/address sanitizer
- Enable bounds checking (default in C++26)
- Replace char/short => uint32, int32 => uint64
  - Check for out-of-bounds values
- Use secure language: Rust (*does not fix all problems*)

# Uncontrolled resource consumption DoS



# Uncontrolled resource consumption DoS




Mitigated by:

- It's complicated...
- Insurance: cloud-based DDoS protection (Cloudflare etc.)
- Insurance: have experts on standby
- Use a reverse proxy/CDN to absorb traffic
- Track and report traffic metrics

# Security fail taxonomy

+ Improper access control: 4 authentication
+ Improper interaction between correctly-behaving entities: e.g.: optimization due to UB
+ Improper control of resource throughout lifetime:
  3 Buffer issues + 5 Path traversal + 6 use after free + 7 dangerous upload + 11 resource DoS
+ Incorrect calculation: 10 integer overflow
+ Insufficient control flow management: e.g.: race condition
+ Protection mechanism failure: e.g.: weak encryption
+ Incorrect comparison: e.g.: permissive list of allowed inputs
+ Improper check or handling of exceptional conditions: 9 NULL pointer
+ Improper neutralization: 1 Cross-site + 2 injection + 8 input validation
+ Improper adherence to coding standards: 9 NULL pointer + missing `break` in `switch`

# Mitre Top 25

| Rank | ID | Name | Score |
|---|---|---|---|
| 1 | 79, 352, 918 | Cross-site tricks: scripting XSS, request forgery CSRF, SSRF | 80.54 |
| 2 | 89, 78, 94, 77, 502 | Injection: SQL, OS, Code, command | 66.12 |
| 3 | 787, 125, 119 | Out-of-bounds RW Buffer overrun, invalid buffer ops | 60.31 |
| 4 | 287, 306, 269, 200, 862, 798 | Improper/missing/hardcoded authentication | 32.54 |
| 5 | 22 | Path traversal ../../.. | 12.74 |
| 6 | 416 | Use after free | 10.19 |
| 7 | 434 | Dangerous file upload | 10.03 |
| 8 | 20 | Improper input validation | 6.78 |
| 9 | 476 | NULL pointer dereference | 3.58 |
| 10 | 190 | Integer overflow | 3.37 |
| 11 | 400 | Uncontrolled resource consumption DoS | 3.23 |

# Bonus: top 10 hardware fails – not ordered

| Rank | Name |
|---|---|
| CWE-1189 | Improper isolation of shared resources on system-on-a-chip (soc) |
| CWE-1191 | On-chip debug and test interface with improper access control |
| CWE-1231 | Improper prevention of lock bit modification |
| CWE-1233 | Security-sensitive hardware controls with missing lock bit protection |
| CWE-1240 | Use of a cryptographic primitive with a risky implementation |
| CWE-1244 | Internal asset exposed to unsafe debug access level or state |
| CWE-1256 | Improper restriction of software interfaces to hardware features |
| CWE-1260 | Improper handling of overlap between protected memory ranges |
| CWE-1272 | Sensitive information uncleared before debug/power state transition |
| CWE-1274 | Improper access control for volatile memory containing boot code |
| CWE-1277 | Firmware not updateable |
| CWE-1300 | Improper protection of physical side channels (Spectre) |